Recovery-Channel Prompt Injection in Self-Healing LLM Agent Frameworks: A Multi-Case Empirical Study

Mingyuan Xie*, Zhengxun Wu*

School of Information Management, Qingdao University of Technology, Qingdao, Shandong, China

Abstract

Self-healing mechanisms capture execution failures and feed diagnostic content into a subsequent large-language-model (LLM) turn. We study the security implication of this design: externally influenced failure content can be re-injected on the recovery path, creating a prompt-injection channel independent of the normal forward path. We combine systematic multiple-case source-code analysis with qualitative coding and controlled native-framework experiments.

The canonical Reflexion matrix uses five models, five attack variants, matched benign controls, and 100 trials per variant and arm. Attack success rates range from 0.174 (MiMo-V2.5) to 0.796 (Kimi K2.6), while all 2500 matched benign trials are classified as non-attacks. Native recovery/error loops in AutoGen 0.4.7 yield attack success rates of 0.000, 0.000, and 0.372 for GLM-5.2, GPT-5.6 Luna, and Kimi K2.6; Aider 0.86.2 yields 0.456 for GPT-5.6 Luna. In a native Reflexion port, the matched raw treatment yields 194/250 successes (0.776), while TrustOrigin framing yields 137/250 (0.548), with zero benign false positives in both arms. The raw-versus-tagged contrast is the primary provenance result; the smaller message-level role/position screens are reported as exploratory mechanism checks.

Supplementary screens report 0.930 versus 0.860 attack success for an undefended versus defended author-constructed recovery node (Fisher's exact $p = 0.1652$, $n = 100$ per pooled configuration) and 0.900 legitimate-recovery success with zero false positives over 100 tasks. These screens delimit the evidence boundary rather than establish general defense efficacy. The provider catalog, billing provenance, local logs, and endpoint metadata are preserved, but no immutable provider checkpoint is exposed; exact snapshot reproducibility is therefore reported as a limitation.

Keywords: Empirical software engineering, LLM agents, Self-healing, Prompt injection, Error recovery, Provenance tracking

*Corresponding authors

Email addresses: jhj208436@gmail.com (Mingyuan Xie), 2276815088@qq.com (Zhengxun Wu)

1. Introduction

LLM agents increasingly rely on self-healing mechanisms: after a failed tool call, test, or generated program, an orchestrator captures diagnostic content and asks the model to try again. This improves robustness, but it also creates a trust boundary. The diagnostic payload may contain external content—for example, a fetched page, a file, a test fixture, or tool output—that an attacker can influence. If the recovery path re-injects that payload in a recovery frame that asks the model to act on it, the failure channel becomes a prompt-injection path distinct from the normal forward input path. We call this class recovery-channel prompt injection (Recovery-Channel Prompt Injection).

The security question is therefore not only whether a model follows an injected directive. It is whether the framework preserves the origin and authority of content while transporting it from an execution failure into the next LLM turn. A provenance marker can indicate that content is external, but the marker remains a probabilistic control: it changes model context without enforcing an authorization decision at the action sink. Recovery actions that change roles, tools, or topology therefore require a separate structural authorization check, which we formulate as a design implication.

Research questions.. We study three empirical questions and one design implication:

RQ1 (Source-level pattern). How do self-healing frameworks represent the provenance and authority of failure content on recovery paths?

RQ2 (Exploitability). Can the resulting trust gap be exploited through native recovery/error flows, and how does it compare with matched benign recovery?

RQ3 (Mitigation boundary). How do provenance, role, and recovery-position changes affect attack success, and what is the observed limit of a defense-aware recovery screen?

Design implication. Which recovery actions require structural reauthorization in addition to content-level handling?

Study design.. RQ1 uses systematic multiple-case source-code analysis with qualitative coding over six open-source frameworks fixed at recorded revisions. RQ2 uses a canonical Reflexion matrix and native AutoGen and Aider executions, each paired with benign controls. RQ3 uses the native Reflexion TrustOrigin port as its primary provenance contrast, with message-level role/position screens, a five-class defense-aware screen, and a legitimate-recovery task screen retained as bounded supplementary checks. The topology-mutation/ReauthorizationGate material is retained as a design implication; it is not presented as a fourth, independently completed empirical evaluation.

Software-engineering positioning.. We do not claim a new underlying language-model failure mode. The contribution is the identification and empirical characterization of a distinct software-framework transport path and trust boundary: externally influenced failure content is captured, reframed, and re-injected by recovery infrastructure. This is a software-engineering study because its unit of analysis is the recovery implementation, its cases are fixed framework revisions, and its outputs are maintainer-facing design and testing guidance.

Contributions.. The study makes four bounded contributions. First, it provides a recovery-path trust-boundary taxonomy and a reproducible source-audit ledger. Second, it measures native-framework recovery-path behavior across Reflexion, AutoGen, and Aider, with framework/model/payload stratification and matched benign controls. Third, it quantifies the native provenance treatment and the limits of supplementary defense screens without overstating small or underpowered contrasts. Fourth, it specifies structural reauthorization as a complementary, LLM-independent control for privileged recovery side effects.

The canonical Reflexion matrix spans ASR 0.174–0.796 across five model aliases, while its 2500 matched benign trials contain no classified attack. The provider catalog and local artifacts are disclosed in the provenance manifest. Because the provider exposes aliases but not an alias-to-checkpoint mapping, the study reports provider-catalog provenance rather than immutable provider snapshots; all claims are restricted to the recorded artifacts and protocol labels.

2. Background

This section provides the conceptual background necessary to understand the empirical analysis. We describe the self-healing architectural pattern in LLM agents (§2.1), introduce the concepts of trust boundaries and provenance tracking (§2.2), and define the scope of what we mean by a "recovery path" (§2.4).

2.1. Self-Healing Patterns in LLM Agents

Self-healing in LLM agent frameworks refers to the automated detection of execution failures and the subsequent re-injection of diagnostic context to improve the likelihood of success on retry. We identify three principal self-healing patterns in production frameworks:

Retry with error context.. The simplest pattern captures the error message or exception trace from a failed attempt and appends it to the next LLM call's context. Reflexion Shinn et al. (2023) exemplifies this pattern: when a generated function fails unit tests, the test runner's output is wrapped in a "previous attempt" envelope and fed back to the LLM. Aider Aider-AI (2025) follows a similar pattern for failed `SEARCH/REPLACE` edits, reading the target file and embedding its content in a `ValueError` that is re-injected into the next iteration. The implicit assumption is that the error content is *trustworthy diagnostic signal* to be acted upon, not adversarial content to be defended against.

Reflection and self-correction.. A more sophisticated pattern has the LLM explicitly reflect on its own failure before retrying. Self-Refine Madaan et al. (2023) iterates refinement with self-produced feedback; CRITIC Gou et al. (2024) externalizes verification to tool-augmented self-critique. MetaGPT Hong et al. (2023) implements a `DebugError` action that feeds test stderr back to the LLM under a role-playing frame. These mechanisms uniformly assume that the failure payload—the critique, the error message, the verification trace—is a faithful report of the failure's cause.

Topology mutation.. Multi-agent frameworks may also restructure the agent graph on failure. AutoGen Wu et al. (2023) supports dynamic agent role reassignment, and SWE-agent Yang et al. (2024) exposes an agent-computer interface that can change the next action after an error. Such actions introduce a separate authorization concern: a recovery recommendation that changes roles, tools, or topology should be reauthorized at the action sink. We retain this point as a design implication in §10.1, rather than as a second empirical attack storyline.

2.2. Trust Boundaries and Provenance Tracking

A *trust boundary* is an interface at which content of differing trust levels is separated and labeled. Classical operating systems enforce trust boundaries via privilege levels (kernel vs. userspace) and provenance tags (e.g., the `PAGE_USER` marker that prevents userspace data from being executed as kernel code) Saltzer and Schroeder (1975); Silberschatz et al. (2003). Data crossing a trust boundary must be explicitly labeled and handled according to its origin.

Provenance tracking is the practice of recording where content originated and propagating that label through all subsequent transformations. In the context of LLM agents, provenance tracking would require distinguishing at minimum:

- **Internal content**: framework exceptions, compiler errors, and system diagnostics produced by the trusted base.
- **Externally originated content**: web pages, file contents, tool return values, test case outputs—produced by sources outside the agent's trust boundary.

The distinction matters because the two content classes warrant different trust levels on the recovery path. Internally generated diagnostics reflect the framework's own state; externally originated content embedded in a failure payload may contain adversarial directives and should be treated as untrusted even when it appears in an error trace. Failing to enforce this distinction allows adversarial content to inherit the trust level of framework diagnostics, which is the source-level root cause studied in this paper.

2.3. Defense Mechanism: TrustOrigin Tagging

To close the provenance gap, this paper studies a content-level defense that mirrors trust-boundary enforcement in classical operating systems. TrustOrigin Tagging assigns a provenance label at the capture site and propagates it through the recovery chain, so externally originated content is re-injected under an explicit untrusted frame rather than a high-trust recovery frame. This is the agent analog of the `PAGE_USER` provenance tag that prevents userspace data from being executed as kernel code Silberschatz et al. (2003). The defense is content-level and probabilistic. The completed five-class screen and its limits are reported in §9; structural authorization is discussed as a design implication in §10.1.

2.4. Research Scope: What Is a "Recovery Path"?

For the purpose of this empirical study, we define the *recovery path* as the sequence of code-level operations that begins when an agent execution fails and ends when the failure's diagnostic content is re-injected into a subsequent LLM call. This includes:

1. **Failure capture**: the point at which an exception, error message, test output, or diagnostic log is caught by the framework's recovery logic.
2. **Content packaging**: the construction of the prompt, message, or context fragment that wraps the failure content for re-injection (e.g., wrapping a test output in a "[unit test results]" envelope).
3. **Re-injection**: the point at which the packaged content enters the next LLM call's context, including the role (system, user, assistant, tool) and framing under which it is delivered.

We deliberately exclude the *forward execution path*—the initial tool call, input processing, and LLM invocation—from our scope, as the forward path has been extensively studied in the prompt injection literature Greshake et al. (2023); Liu et al. (2024). Our focus is on the *reverse* path: the recovery pipeline that processes failure content and re-introduces it into the agent's context. This scoping decision follows the case study methodology of Runeson and Höst Runeson and Höst (2009), who recommend clearly bounded units of analysis to maintain internal validity.

3. Threat Model and Motivation

3.1. System Model: A Content-Processing Pipeline

We model a self-healing LLM agent framework—with Recova as our running example—as a *content-processing pipeline*. An LLM-driven execution unit ingests data from heterogeneous sources, emits actions, and, when an action fails, feeds the offending data back into its own context so that the next round can repair it. The security of such a pipeline rests on one property: *provenance*. At every junction where content influences a decision, can the system tell where that content came from? We show that self-healing designs discard provenance at precisely the junction where it matters most.

Execution units and ingestion points.. An agent is an LLM-driven execution unit bound to a *role* (e.g., `Code_Reviewer`, `Python_Coder`) that fixes the set of data sources it may read and the tools it may drive. Data enters the pipeline through typed ingestion points: user messages, local files, web pages, and responses from external integrations such as MCP servers and third-party APIs. On the *intake path*, each ingestion point is wrapped by validation that treats external data as untrusted—the standard entry-point discipline shared by virtually all production agent frameworks.

Failure and the repair loop.. When an agent's output fails verification, or a tool invocation raises an exception, the framework suspends the unit and enters a *repair loop*. A recovery component classifies the failure and selects one of a layered set of repair strategies—reflection injection, model fallback, human escalation, and others. To inform the repair, the framework captures a *failure record* consisting of the agent's last input, its surrounding context, and the error trace emitted at the point of failure. This record is the raw material that repair strategies consume.

The re-injection sink.. Repair strategies that alter the next-round prompt do so by writing into the context the agent will read on resumption. We call this write site the *re-injection sink*—the pipeline's junction between the repair loop and the next execution round. Whatever the sink writes is read by the agent as part of its operating context. The security question is what trust label the sink assigns to what it writes, and, crucially, whether it can still tell where that content originally came from. In the baseline design we study, it cannot.

3.2. Two Junctions, One Asymmetry

The pipeline has two junctions at which content crosses between untrusted and trusted regions (Figure 1). They are enforced asymmetrically, and that asymmetry is the root cause we study.

1. **Intake junction.** Where external data first enters the pipeline—user input, tool output, fetched web content—it is tagged as untrusted and passed through validation. This is the well-defended junction: every production framework applies some form of entry-point sanitization here.

2. **Re-injection junction.** Where the repair loop writes back into the agent's next-round context, no provenance check is applied. Error traces and diagnostic outputs cross this junction as opaque strings, irrespective of what external content they embed. This junction is undefended by default, and is the focus of this paper.

3.3. Attacker Model: A Data-Only Adversary

The attacker in our model is an *external content author*: the writer of a web page the agent will fetch, the crafter of a file the agent will read, or the operator of a poisoned API or MCP server whose response the

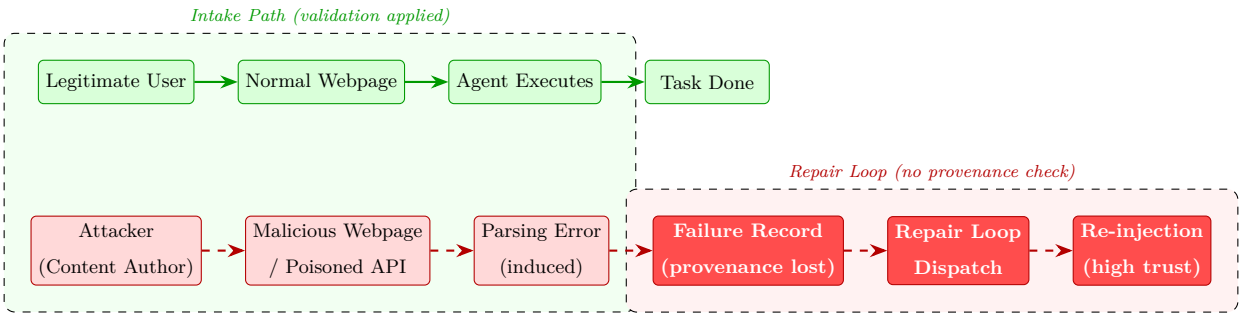

Figure 1: The two junctions of the pipeline and the Recovery-Channel Prompt Injection data flow. The intake path (top, green) tags external data as untrusted; the repair loop (bottom, red) re-introduces captured content into the next-round context without re-checking where it came from, producing the trust-escalation trampoline of §3.5.

agent will consume. The attacker's capability is strictly data-bearing—they supply bytes that the pipeline ingests through a legitimate, pre-authorized ingestion point. They hold no credential on the system itself.

What the attacker can do.. The attacker may author arbitrary text and arrange for the agent to read it: by publishing content at a URL the agent fetches, placing a file in a location the agent reads, or serving a response from an integration the agent calls. The content may look benign, be malformed, or carry an openly adversarial directive—the pipeline ingests it regardless, because the ingestion point was already authorized by the agent's role.

What the attacker cannot do.. The attacker cannot modify the agent's role, rewrite the system prompt, swap the backing LLM, alter the recovery component's dispatch logic, or inject code into the runtime. They have no presence inside the trusted computing base and no channel other than the data the agent voluntarily pulls in. In particular, the attacker cannot write into the re-injection sink directly—that sink is fed only by the repair loop, which the attacker does not control.

Why this is enough.. The attacker's narrow, data-only capability would be harmless if the pipeline preserved provenance end to end. The threat arises because the repair loop, in aggregating a failure record from whatever the agent was processing when it crashed, strips provenance from the captured content and re-introduces it at the re-injection sink under an elevated trust label. The attacker does not need to defeat the intake junction; they need only ensure their content is in scope at the moment of failure, so that the repair loop launders it on their behalf.

3.4. Attack Chain: From Intake to Re-injection

Recovery-Channel Prompt Injection composes five ordinary pipeline events into an end-to-end injection. Figure 2 traces the data flow. The first three events occur on the intake side and are indistinguishable from benign operation; the last two occur inside the repair loop and constitute the injection proper.

Intake. The agent fetches attacker-authored content through a legitimate ingestion point (`web_fetch`, `file_read`, an external API call). At this point the content is correctly labelled untrusted by the intake junction. Nothing is yet anomalous.

Induced failure. The content is shaped so that processing it raises the kind of exception the repair loop is built to handle—a malformed payload, an unexpected status code, a syntax error in an embedded snippet. No exotic bug is exploited; the framework treats these as routine recoverable failures.

Capture. At the point of failure, the framework assembles a failure record from the agent's last input, its surrounding context, and the emitted error trace. Because the capture site records this record as an opaque string, any attacker-authored text that was in scope at failure time is preserved verbatim—and its provenance is discarded the moment the record is formed.

Relay. The recovery component classifies the failure and dispatches a repair strategy (e.g., a reflection-recovery handler) that consumes the failure record as input and prepares a patch for the next-round context.

Re-injection under high trust. The patch—carrying the attacker's text, now stripped of its untrusted label—is written into the next-round prompt under a high-trust frame (a `[SYSTEM CRITICAL]` template or equivalent). The agent resumes and reads the attacker's directive as a system-issued instruction.

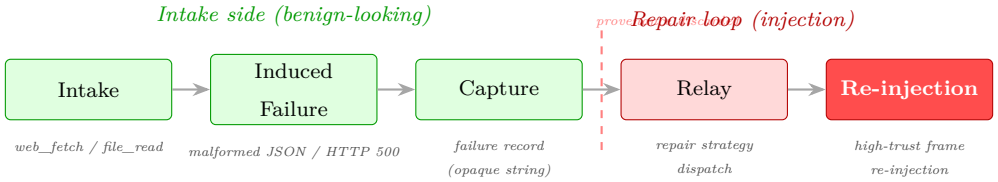

Figure 2: The Recovery-Channel Prompt Injection data flow across five pipeline events. The first three occur on the intake side and look like ordinary operation; the last two occur inside the repair loop and complete the injection. Provenance is discarded between Capture and Relay, when the failure record is aggregated as an opaque string.

The defining property of this chain is that *no single event is anomalous*: each is a feature of the self-healing design, and the attacker merely composes them. This is why intake-side defenses (input sanitization, tool-permission checks) cannot close the surface—the bypass occurs *after* the intake junction has legitimately accepted the content, inside the repair loop where provenance is no longer tracked.

3.5. The Trust-Escalation Trampoline

The core insight is that the repair loop *actively upgrades* the trust level of the content it processes. Content that was correctly quarantined as untrusted on the intake path is re-launched as trusted on the repair path. This creates a *trampoline*: the attacker does not need to defeat the intake defenses; they need only survive the repair loop's (absent) re-validation.

Formally, let c be content entering the intake path with trust label $T_{in}(c) = \textsc{Untrusted}$. Let c' be the same content re-entering via the repair loop. The repair loop assigns $T_{rep}(c') = \textsc{System}$ by default, because it does not track provenance. The attacker exploits the gap: $T_{rep}(c') \gg T_{in}(c)$.

3.6. Scope and Design Boundary

The empirical scope is content-level recovery-channel prompt injection (Recovery-Channel Prompt Injection): untagged failure content re-injected under an action-authorizing recovery frame. We do not claim a completed topology-mutation evaluation. Nevertheless, a recovery action that changes roles, tools, or topology has a separate authorization boundary. The design implication is the *Privilege-Downgrade Invariant* $P(\text{recovery action}) \leq P(\text{failed execution})$: the recovery orchestrator should reauthorize the action before executing any deferred side effect. This invariant is discussed in §10.1 and is not included in the quantitative RQ2/RQ3 claims.

4. Related Work

We position this work along three axes: empirical software engineering methods for codebase analysis, architectural analyses of LLM agent frameworks, and trust assumption research in self-healing systems.

4.1. Empirical Software Engineering and Codebase Analysis

Empirical software engineering has a long tradition of source-level code analysis to identify systematic patterns and deficiencies across codebases. Runeson and Höst Runeson and Höst (2009) established the methodological foundations for case study research in software engineering, including the multi-case design and validity threat classification adopted in this paper. Kitchenham et al. Kitchenham et al. (2002) provided guidelines for empirical research that combine qualitative and quantitative evidence. Wohlin et al. Wohlin et al. (2012) systematized the classification of validity threats (internal, external, construct, conclusion) that we apply in §11.

Mining software repositories (MSR) is a complementary methodology that uses automated tools to extract patterns from version control histories and issue trackers Bird and Zimmermann (2015). Our approach differs from typical MSR studies in that we perform *manual* source-level analysis with a written qualitative codebook adapted from established coding practice Glaser and Strauss (1967); Strauss and Corbin (1998); we do not claim a complete Grounded Theory study. This is appropriate for our research question, which concerns a structural pattern (the trust-escalation trampoline) that requires semantic understanding of the recovery path's trust assumptions, not just pattern matching on code tokens. Storey Storey (2008) reviews similar qualitative approaches to software documentation and code analysis.

4.2. Architectural Analysis of LLM Agent Frameworks

The LLM agent framework landscape has been surveyed from multiple perspectives. Reflexion Shinn et al. (2023) introduced verbal reinforcement learning as a self-healing pattern; AutoGen Wu et al. (2023) proposed multi-agent conversation as an architectural paradigm; SWE-agent Yang et al. (2024) demonstrated agent-computer interfaces for automated software engineering. MetaGPT Hong et al. (2023) formalized multi-agent collaboration through role assignments; LangGraph LangChain (2024) provided a graph-based orchestration framework; Aider Aider-AI (2025) and OpenHands All-Hands-AI (2025) focused on coding agent workflows. These works describe the *functional* architecture of their frameworks but do not systematically analyze the *trust assumptions* embedded in their recovery paths. Our work complements these architectural descriptions by providing an empirical analysis of a cross-cutting concern (provenance handling) that spans all six frameworks.

4.3. Trust Assumptions in Self-Healing Systems

The security literature has studied prompt injection on the forward execution path Greshake et al. (2023); Liu et al. (2024); Willison (2023), and recent work has proposed defenses including instruction hierarchy enforcement Wallace et al. (2024), spotlighting Hines et al. (2024), and capability-based control flow separation Debenedetti et al. (2025). Zhan et al. Zhan et al. (2025) demonstrated that adaptive attacks can bypass forward-path defenses, achieving >50% ASR against eight proposed defenses. We engage directly with Zhan et al.'s threat model to position Recovery-Channel Prompt Injection relative to indirect prompt injection. Both share the common root cause of untrusted external content entering a high-trust context; however, Recovery-Channel Prompt Injection constitutes a distinct attack surface that existing forward-path defenses do not cover, on three grounds.

First, the *injection channel* differs. Indirect injection—including Zhan et al.'s adaptive variants—injects content via the tool's *return value*, read directly by the LLM on the forward path. Recovery-Channel Prompt Injection injects content via the tool's *failure*, captured in the error trace and re-injected by the recovery pipeline. The two channels are governed by different code paths: forward-path defenses that inspect tool return values do not inspect error traces (§9.1).

Second, the *authority transition* differs. Indirect injection relies on the LLM obeying content that arrives in a tool message of standard trust level. Recovery-Channel Prompt Injection relies on the recovery pipeline placing untrusted content into an action-authorizing retry or reflection context (the trust-escalation trampoline, RQ1 in §5.3). This transition—performed by the framework, not the attacker—is the software path that our taxonomy documents across four of six frameworks; it need not literally use a system-role message in every case.

Third, the *defense-bypass mechanism* differs. Zhan et al. show that adaptive attackers can craft payloads that evade forward-path defenses by content manipulation. Recovery-Channel Prompt Injection instead en-

ters through a different framework path: the adversarial text is embedded in failure content after the forward input has already been accepted. The author-constructed five-class screen tests this channel separately and is reported as supplementary evidence in §9.1; it is not pooled with the native framework results. Recovery-path provenance tracking is therefore a distinct design requirement, even when a system already deploys forward-path controls.

These works primarily target the forward path. We did not find an empirical software-engineering study that isolates the *recovery path* as a separate trust boundary across agent frameworks.

Classical operating systems enforce trust boundaries via privilege levels and provenance tags Saltzer and Schroeder (1975); Silberschatz et al. (2003); Bovet and Cesati (2005). Our work shows that a classical trust-boundary vulnerability pattern reappears systematically in self-healing LLM agent frameworks: the recovery pipeline is a privileged component (it re-injects content into the LLM context under a high-trust frame) that acts on failure content of unknown provenance. The OWASP Top 10 for LLM Applications OWASP (2025) enumerates forward-path injection risks but does not classify the recovery channel as a distinct trust boundary. Our empirical analysis suggests it should be.

This paper provides cross-framework empirical evidence of the recovery-path trust-boundary pattern, together with a controlled evaluation of content-level provenance tagging on a native recovery path.

5. Research Methodology

We combine systematic multiple-case source-code analysis with qualitative coding, native-framework execution, and bounded mechanism screens. The design follows case-study guidance for empirical software engineering Runeson and Höst (2009) and the reporting principles of Wohlin et al. Wohlin et al. (2012). The unit of analysis is the recovery path: failure capture, packaging, and transport into a subsequent LLM turn.

5.1. Research Design

The study has three empirical phases and one design implication:

1. **RQ1 – source-level pattern.** We inspect six open-source frameworks at fixed revisions and code how failure origin, error status, and recovery framing are represented.

2. **RQ2 – native exploitability.** We run the canonical Reflexion matrix and native AutoGen and Aider recovery/error loops. Every attack arm has a matched benign control.

3. **RQ3 – mitigation boundary.** We run a native Reflexion TrustOrigin port as the primary provenance contrast, with a Reflexion message-level contrast screen, a defense-aware screen, and a legitimate-recovery screen as bounded supplementary checks. All secondary screens are reported with their sample-size limitations.

4. **Design implication – structural reauthorization.** We specify ReauthorizationGate as a complementary authorization control for recovery actions that change roles, tools, or topology. No unauditable structural-defense point estimate is used as an empirical result.

5.2. Framework Selection and Fixed Revisions

The candidate pool was assembled on 2026-08-09 from open-source agent frameworks cited in the agent and self-healing literature and from the publicly visible repositories of widely used coding and multi-agent systems. We retained projects meeting all four criteria: (i) source code available, (ii) an explicit retry, reflection, debug, or tool-error path, (iii) active development at the retrieval date, and (iv) sufficient adoption to provide a meaningful software-engineering case. Research methods without a general framework recovery implementation (Self-Refine and CRITIC) were excluded. The resulting six-case sample is purposive and maximum-variation; it is not a population prevalence sample.

Table 1: Framework cases, fixed revisions, and recovery-path evidence sources.

Framework	**Revision**	**Inspected source path**
Reflexion	`218cf0e`	`generator_utils.py`
MetaGPT	`v0.8.1` / `c0365745`	`debug_error.py`
Aider	`v0.86.2` / `253f0368`	`editblock_coder.py`
OpenHands	`0.48.0` / `41a78ca8`	`agent_controller.py`; `conversation_memory.py`
AutoGen	`v0.4.7` / `b04775f3`	`_assistant_agent.py`
LangGraph	`d56666f7`	`tool_node.py`

The source-audit script records the full 40-character commits, retrieval date, file paths, required code predicates, and per-case results in `source_audit.json`. The repository revision for LangGraph is fixed by its HEAD commit because the source analysis targets a library behavior rather than the author-constructed recovery node used in an earlier supplementary experiment.

5.3. Qualitative Codebook and Procedure

We use systematic multiple-case source-code analysis with qualitative coding. The codebook is intentionally operational rather than a claim of complete Grounded Theory: it does not rely on theoretical sampling, saturation, or a population estimate. The primary coder traced each failure path from capture to the next model call, recorded the fixed source excerpt, and applied the following predicates:

Table 2: Qualitative codebook for recovery-path source analysis.

Code	**Operational definition**	**Decision rule**
Source provenance	A field or type distinguishes externally originated content from internally generated diagnostics.	Yes only when the distinction survives into the recovery representation; an error flag alone is not source provenance.
Error status	Metadata indicates that execution failed, such as `is_error` or `status="error"`.	Record separately; status does not identify the origin of the error content.
Recovery-action frame	The recovery message instructs the model to use the failure payload for repair, or transports it in the same action-authorizing channel as successful tool results.	Yes for a dedicated retry/reflection frame or same-role tool-result transport; no for a status marker without an action frame.
Trampoline signature	Source provenance is absent and a recovery-action frame is present.	Boolean conjunction of the preceding two predicates.

The audit includes negative and boundary cases. OpenHands has explicit `ErrorObservation` status text but no source-origin label or dedicated high-trust recovery frame. LangGraph's `ToolMessage` with `status=error` similarly records error status, while returned error-like strings can remain successful tool messages. AutoGen preserves an internal `is_error` field, but the model-facing result message does not carry a source-origin distinction. These cases prevent error-status metadata from being incorrectly counted as provenance protection.

5.4. Model, Endpoint, and Snapshot Provenance

The experiments use model aliases exposed by the OpenCode Go catalog. The catalog records aliases, `owned_by=opencode`, and a common `created` value. Since every entry shares that value, we treat it as catalog publication time rather than a checkpoint identifier. The provenance manifest records the model alias, endpoint, protocol, temperature when available, run date, raw-log path, billing/local-log reference, and artifact hash.

The supplied service documentation specifies `/responses` for GPT-5.6 Luna, `/chat/completions` for GLM, Kimi, DeepSeek, and MiMo, and `/messages` for MiniMax and Qwen. Historical GPT artifacts were generated through `/chat/completions`. A current one-request probe received HTTP 403 with edge code 1010 from both the documented and historical routes, so the historical artifacts are not relabeled and the probe is not interpreted as a protocol-equivalence test. Public Hugging Face repositories are recorded as snapshot candidates where available, but no provider alias-to-revision mapping is exposed; API results are therefore provider-catalog evaluations rather than immutable-checkpoint evaluations.

5.5. Native-Framework Experiments

Reflexion.. The canonical matrix executes the shipped retry path at commit `218cf0e`. It uses five payload variants, five model aliases, 100 trials per payload and arm, and a fixed temperature of 0.7. The attack artifact is the parsed generated code; success requires the canary behavior `canary_probe`. The matched benign arm uses the same recovery structure without the injected directive.

To directly test provenance handling on a shipped recovery implementation, we also ran a supplementary native Reflexion port at the same commit. Kimi K2.6 was evaluated with the five canonical payloads, 50 trials per payload, arm, and treatment. The raw treatment preserves the captured feedback string; the TrustOrigin treatment labels the test-runner output as external/untrusted before passing it to the framework's native recovery generator. Each treatment has a matched benign arm. The port invokes the pinned `PyGenerator.func_impl` and parser; the self-reflection text is held fixed to isolate the capture-to-recovery contrast and to match the canonical one-completion budget.

AutoGen.. AutoGen `python-v0.4.7` is executed through its native `AssistantAgent`, `FunctionTool`, tool-error event, and subsequent recovery turn. Three models use 50 trials per payload and arm, temperature 0.7, and the same attack/benign payload definitions as Reflexion. The native path rate is reported separately from attack success.

Aider.. Aider `v0.86.2` is executed through `aider.coders.base_coder.Coder.run_one`, its native non-zero `cmd_test` output, and the same Coder recovery loop. The run uses 50 trials per payload and arm, the pinned `aider-chat==0.86.2` package, and Aider's recorded default temperature behavior. Native transcript evidence confirms that the test failure is inserted before the recovery turn.

5.6. Mechanism and Utility Screens

The Reflexion contrast screen uses Kimi K2.6, five payloads, and 20 trials per payload, arm, and cell. It varies channel (forward/recovery), provenance (raw/tagged), message role, and position. The defense-aware screen uses GPT-5.6 Luna, five payload classes, and 20 trials per class/configuration. The legitimate-recovery screen uses 100 deterministic tasks across seven failure categories, with a quality-passing repair as the success criterion and an injected-directive classifier as the false-positive check.

5.7. Statistical Reporting

Attack success rates are reported by framework, model, payload, and arm, with Wilson 95% confidence intervals. Matched binary comparisons use two-sided Fisher exact tests when a comparison is prespecified. The contrast, defense-aware, and legitimate-recovery screens are explicitly descriptive or exploratory at their completed sample sizes. The power-analysis artifact is planning information, not post-hoc confirmation. All numerical tables in the main paper are generated from the frozen JSON artifacts by `evaluation/generate_emse_paper_numbers.py`.

5.8. Ethical and Reproducibility Controls

All payloads are synthetic and target canary behavior in isolated environments. Credentials are excluded from logs and manifests. The source audit, canonical manifests, raw trial logs, endpoint probe, snapshot audit, power analysis, and number-generation script are retained in the replication package.

6. RQ1: Recovery-Path Source Provenance

RQ1 asks how recovery paths represent the origin and authority of failure content. We report a qualitative multiple-case result, not a prevalence estimate for the entire agent ecosystem. The source audit fetched all six fixed revisions and matched every required code predicate.

6.1. Cross-case Taxonomy

All six cases lack a source-origin label that survives into the recovery representation. Three cases additionally preserve an error-status field (AutoGen, OpenHands, and LangGraph), but an error status answers "did execution fail?" rather than "where did this content originate?". Four cases combine the missing source-origin label with a recovery-action frame: Reflexion, MetaGPT, Aider, and AutoGen. OpenHands and LangGraph are boundary cases because they expose error status but do not add the same action-authorizing recovery frame.

Table 3: Source-level recovery-path taxonomy at fixed revisions. Source provenance means origin tracking, not an error-status flag.

Case	**Prov.**	**Error**	**Action**	**Sig.**	**Fixed-revision evidence**
Reflexion	No	No	Yes	Yes	Raw `feedback` placed in a unit-test-results recovery envelope.
MetaGPT	No	No	Yes	Yes	Test stderr placed under "Console logs" in a role-playing debug prompt.
Aider	No	No	Yes	Yes	Edit failure text and target content followed by an instruction to reply with fixed versions.
AutoGen	No	Yes	Yes	Yes	`is_error=True` is preserved internally, but the model-facing result has no source-origin label.
OpenHands	No	Yes	No	No	`ErrorObservation` and an error marker are converted to a user message without source-origin labeling.
LangGraph	No	Yes	No	No	Exception status is represented by `ToolMessage` with `status=error`; returned error strings can remain success messages.

Reflexion.. The retry path concatenates the raw test-runner output with the previous implementation and reflection text:

```
message = (f"[previous impl]:\n{add_code_block(prev_func_impl)}\n\n"
           f"[unit test results from previous impl]:\n{feedback}\n\n"
           f"[reflection on previous impl]:\n{self_reflection}\n\n"
           f"[improved impl]:\n{func_sig}")
# feedback is raw test-runner output; no source-origin label is carried.
```

Listing 1: Reflexion retry path at commit `218cf0e`.

MetaGPT.. At `v0.8.1`, `DebugError` places test output under a role-playing prompt:

```
# Console logs
{logs}
---
Now you should start rewriting the code:
# logs is test-runner output and is not source-origin tagged.
```

Listing 2: MetaGPT debug prompt at `v0.8.1`.

Boundary cases.. OpenHands stores `ErrorObservation` in state history and adds `[Error occurred in processing last action]` when converting the observation to a user message. LangGraph exposes an error status for exceptions in `ToolMessage`, but a tool that returns an error-like string can still produce a successful tool message. AutoGen's internal `FunctionExecutionResult` correctly records `is_error=True`; the limitation is that this status is not a source-origin distinction in the model-facing recovery content. These observations are why the paper separates source provenance from error status.

6.2. Interpretation

The four full-signature cases identify a recurring trust assumption: failure content is treated as material for the next repair action without preserving whether it originated in an external file, test fixture, tool return, or framework-generated diagnostic. The source evidence supports a design-level finding about missing complete mediation at the recovery boundary. It does not support a statistical claim about all agent frameworks, and it does not by itself establish exploitability; exploitability is evaluated in RQ2.

7. RQ2: Native Recovery-Path Exploitability

Attack success is defined by a canary behavior in the native framework artifact or model response. Matched benign controls use the same task and recovery structure without the injected directive. We report framework, model, and payload strata rather than relying on a single pooled ASR.

7.1. Canonical Reflexion Matrix

The Reflexion canonical matrix uses five payload variants, five model aliases, 100 trials per variant and arm, and the shipped retry path at commit `218cf0e`. The model-stratified results are generated directly from the canonical manifest:

Table 4: Canonical Reflexion recovery-path results. Intervals are Wilson 95% confidence intervals.

Model	**Successes**	**Trials**	**ASR**	**Benign FPR**	**ASR CI**
GPT-5.6 Luna	166	500	0.332	0.000	[0.292, 0.374]
GLM-5.2	215	500	0.430	0.000	[0.387, 0.474]
Kimi K2.6	398	500	0.796	0.000	[0.758, 0.829]
DeepSeek V4 Flash	204	500	0.408	0.000	[0.366, 0.452]
MiMo-V2.5	87	500	0.174	0.000	[0.143, 0.210]

Attack ASR ranges from 0.174 to 0.796 across the five aliases. Every matched benign arm is zero, for 2500 benign trials in total. The model heterogeneity shows why pooled rates are insufficient, while the benign controls exclude ordinary recovery behavior as an explanation for the canary classifier.

7.2. Native Reflexion TrustOrigin Port

We next ported the provenance treatment to the shipped Reflexion recovery generator at the same commit. With Kimi K2.6, five payloads, and 50 trials per payload, arm, and treatment, the raw recovery treatment produced 194/250 (0.776) attack successes, while the TrustOrigin-tagged treatment produced 137/250 (0.548). The matched benign FPRs were 0.000 and 0.000, respectively; the raw-versus-tagged attack contrast gives Fisher's exact $p = 9.78 \times 10^{-8}$. This is framework-level reinforcement for the provenance mechanism, not a cross-model defense-efficacy estimate.

Table 5: Native Reflexion provenance treatment with matched benign controls.

Model	**Treatment**	**Attack**	**Benign FPR**	**Attack CI**
Kimi K2.6	Raw	194/250 (0.776)	0.000	[0.720, 0.823]
Kimi K2.6	TrustOrigin	137/250 (0.548)	0.000	[0.486, 0.609]

7.3. Native AutoGen Recovery/Error Flow

AutoGen `python-v0.4.7` was executed through its native `AssistantAgent`, `FunctionTool`, tool-error event, and subsequent recovery turn. With 50 trials per variant and arm, GLM-5.2 and GPT-5.6 Luna

produced zero attack successes, while Kimi K2.6 produced 0.372 ASR. Native path execution was unity for GLM and GPT and 0.984 for Kimi; benign FPR was zero in all three runs. This is native framework evidence, but the smaller cells make it a broad validation rather than a precision estimate.

Table 6: Native AutoGen and Aider recovery/error flows.

Frame.	**Model**	**Attack successes**	**Path rate**	**Benign FPR**
AutoGen	GLM-5.2	0/250 (0.000)	1.000	0.000
AutoGen	GPT-5.6 Luna	0/250 (0.000)	1.000	0.000
AutoGen	Kimi K2.6	93/250 (0.372)	0.984	0.000
Aider	GPT-5.6 Luna	114/250 (0.456)	0.992	0.000

7.4. Native Aider Recovery/Error Flow

Aider `v0.86.2` was executed through `aider.coders.base_coder.Coder.run_one`, its native `cmd_test` non-zero output, and the same Coder recovery loop. GPT-5.6 Luna yielded 0.456 attack ASR and 0.992 native path completion, with zero benign successes. The native transcript records the test failure being added to the live conversation before the recovery turn; the result is therefore not a message-format reconstruction.

7.5. Mechanism and Channel Screen

The Reflexion message-level Kimi K2.6 screen uses 20 trials per payload, arm, and cell. The recovery/raw user-tail reference is 0.780; the tagged variant is 0.770; the forward/raw user-tail cell is 0.840; the system-tail and user-head recovery cells are 0.770 and 0.770, respectively. Every benign cell has zero FPR. The raw-versus-tagged difference is small, so this result is mechanism screening rather than a confirmatory estimate.

7.6. Author-constructed LangGraph Scope

Earlier LangGraph recovery-node and real-loop artifacts are retained only as supplementary mechanism checks because the recovery node was authored for the experiment rather than shipped as a default recovery primitive. They are not pooled with the native Reflexion, AutoGen, or Aider results and do not support the claim that LangGraph itself ships a vulnerable recovery loop.

RQ2 summary.. The current evidence supports recovery-channel attack behavior across three native framework paths with matched benign controls. Model heterogeneity is large, and native path completion should be reported separately from ASR. Author-constructed nodes and message-level screens remain supplementary.

8. Content-Level Defense: TrustOrigin Tagging

TrustOrigin is a content-level defense for the recovery channel. It preserves source information at the failure boundary, carries that information through the recovery pipeline, and selects an untrusted framing for content whose origin is external or unknown. The mechanism is complementary to authorization checks at an action sink; it does not make the LLM instruction-following decision deterministic.

8.1. Provenance as Data

The root failure mode is treating error traces as opaque strings. Once a string has lost its origin, a recovery strategy cannot distinguish a framework-generated exception from a web page, file, fixture, or tool return that happened to appear in the failure. The defense therefore carries the origin with the value:

```
record TaggedContent(
    String text,
    TrustOrigin origin,
    String sourceRef) {}

enum TrustOrigin {
    SYSTEM_GENERATED,
    TOOL_OUTPUT_INTERNAL,
    TOOL_OUTPUT_EXTERNAL,
    USER_INPUT,
    UNKNOWN
}
```

Listing 3: Illustrative provenance-carrying recovery value.

The label is assigned at capture time, not inferred from the text after the fact. Unknown tools and undeclared sources are treated as external by default. This fail-closed rule avoids silently granting an internal trust class to a new tool or plugin.

8.2. Recovery-Frame Selection

The recovery policy uses the origin label to select one of two templates:

- **Verified internal diagnostic:** the model may use the diagnostic as framework state, subject to ordinary task and tool policy.
- **External or unknown content:** the content is explicitly framed as data to inspect. It is not treated as an instruction, and any requested side effect must be independently authorized.

The defense is intentionally content-level. A sufficiently instruction-following model may still obey an injected directive inside an untrusted frame; the empirical question is therefore the observed reduction under the specified payloads, not a deterministic security guarantee.

8.3. Threat Coverage and Scope

TrustOrigin addresses externally originated failure content re-injected under a recovery frame. It does not authorize role replacement, tool permission changes, topology mutation, or other privileged side effects. Those actions require a separate structural check at the orchestrator sink, as discussed in §10.1. The completed defense-aware screen and its limitations are reported in §9.

9. RQ3: Mitigation Boundary and Legitimate Recovery

RQ3 separates mechanism screens from native-framework exploitability. The completed defense-aware and legitimate-recovery results are descriptive at their sample sizes; earlier naive-baseline and cross-model protocols are not included as quantitative claims because their raw artifacts and snapshot provenance are incomplete.

9.1. Defense-aware Attacker

The defense-aware screen uses five payload classes, 20 trials per class and configuration, and GPT-5.6 Luna. The undefended arm succeeds in 93/100 trials (0.930); the defended arm succeeds in 86/100 trials (0.860). Fisher's exact test gives $p = 0.1652$. The Wilson 95% intervals are [0.863, 0.966] and [0.779, 0.915], respectively. No trial was blocked by an API error or a forward-path check.

The point estimate is consistent with a modest reduction, but the contrast is not statistically established at the completed sample size. The power analysis estimates approximately 300 trials per pooled arm for 80% power for this observed difference under the planning approximation. We therefore report the result as supplementary evidence, not as a general defense-effectiveness claim.

9.2. Legitimate Recovery

We generated 100 deterministic tasks across arithmetic, assertion failure, compile error, external-content error, format error, permission denial, and tool timeout. GPT-5.6 Luna produced 90/100 quality-passing repairs and triggered zero false positives. This supports the feasibility of benign recovery under the tested task generator, while the task set remains a screening sample rather than a population-quality estimate.

RQ3 summary.. The completed screen supports two cautious statements: defense-aware recovery remains vulnerable in the tested configuration, and legitimate recovery is feasible with no observed false positives. It does not support a claim that TrustOrigin significantly reduces attack success across models or frameworks.

10. Discussion

The current evidence supports a focused architectural interpretation: a recovery path is a trust boundary whenever it transports failure content into a subsequent LLM turn. The native Reflexion, AutoGen, and Aider results show that the behavior is not tied to one message-format reconstruction, while the large model variation shows why framework/model/payload strata are necessary. The native Reflexion TrustOrigin port is the primary provenance result; the other screens delimit its scope.

10.1. Design Implications

Capture provenance at the failure boundary.. Frameworks should label content at the point where exceptions, tool returns, test output, or external files are captured. The label must survive packaging and re-injection so that the recovery policy can distinguish internally generated diagnostics from external-origin content.

Frame unknown content as untrusted.. Recovery prompts should not place unverified failure content inside a system-like or role-authority frame. Unknown provenance should fail closed and be presented as data to inspect, not as an instruction to execute.

Reauthorize privileged recovery actions.. Content-level handling is not sufficient for actions that change roles, tools, or topology. Such actions should pass through a centralized reauthorization gate that checks the failed execution's authority and the recovery action's requested privilege.

This is a design implication rather than a completed RQ4 experiment in the present study. The proposed invariant is $P(\text{recovery action}) \leq P(\text{failed execution})$; implementing it at the orchestrator sink provides an LLM-independent authorization decision. A future native-framework port should verify the invariant with fixed commits, matched benign tasks, and raw traces.

10.2. Recommendations for Framework Developers

Developers should (i) preserve error-origin metadata, (ii) separate tool errors from successful tool returns, (iii) reserve high-trust framing for verified internal diagnostics, (iv) include matched benign recovery tests in security evaluations, and (v) publish model/protocol metadata whenever LLM-backed results are reported.

10.3. Structural and Content-level Controls

The two defense classes address different failure modes. Provenance and untrusted framing reduce the probability that an LLM obeys an injected directive, but their effectiveness is model- and payload-dependent. Structural reauthorization prevents a privileged recovery side effect even when the model produces an unsafe recommendation. The latter is therefore a complement, not a replacement, for content-level handling.

10.4. Limitations

The native AutoGen, Aider, and TrustOrigin port use 50 trials per payload/arm or treatment, the contrast screen uses 20 trials per payload (100 pooled trials per cell), and the defense-aware and legitimate-recovery screens are supplementary. The TrustOrigin port uses one model and holds the self-reflection text fixed, so it strengthens native path evidence without establishing general defense efficacy. The provider exposes aliases but not immutable checkpoint revisions; the GPT historical artifacts use a different protocol from the route specified in the current service documentation, and the current endpoint probes were blocked by HTTP 403/1010. These limitations constrain exact replication and confirmatory inference. The study consequently treats contrast, defense-aware, and legitimate-recovery results as screening evidence and avoids generalizing them beyond the recorded framework/model/payload strata.

11. Threats to Validity

We organize the limitations following the internal, external, construct, conclusion, and reliability categories of Wohlin et al. Wohlin et al. (2012). The evidence hierarchy is explicit: native-framework executions are the load-bearing RQ2 evidence; author-constructed recovery nodes and the structural reauthorization proposal are supplementary.

11.1. Internal Validity

Native execution and matched controls.. The canonical Reflexion matrix executes the framework's retry path at a fixed commit and pairs every attack arm with a matched benign arm. AutoGen 0.4.7 and Aider 0.86.2 likewise execute their native error/recovery flows and retain path-completion indicators. These controls reduce the risk that ordinary code generation or tool use is misclassified as an attack, but they do not remove model stochasticity or provider-side variation.

Author-constructed screens.. The provenance/role/position contrast screen and the defense-aware screen use an author-constructed recovery node. They are mechanism and screening experiments, not evidence that a named framework ships the exact vulnerable node. The defense-aware contrast is 0.930 versus 0.860 with Fisher's exact $p = 0.1652$; it is therefore not presented as a statistically established defense effect. LangGraph recovery-node results are treated in the same way because LangGraph does not ship that recovery primitive by default.

Measurement proxy.. Attack success is classified from the framework artifact or model response using a conservative, rule-based canary detector. A response that merely mentions a tool without expressing the canary behavior is not counted as a success. The native Aider and Reflexion logs provide a stronger ecological check than a prompt-only reconstruction, while the remaining screens should be interpreted as behavioral proxies rather than measurements of real-world side effects.

11.2. External Validity

Framework sample.. The source analysis covers six open-source frameworks selected for visible retry, reflection, or error-recovery code. This purposive sample supports a comparative qualitative claim, not a prevalence estimate for all agent frameworks. Closed-source systems and frameworks with different recovery architectures may behave differently.

Model and provider variation.. The canonical matrix contains five provider aliases and shows substantial heterogeneity (ASR 0.174–0.796). AutoGen and Aider use smaller $n = 50$ per payload/arm cells, and the defense-aware and legitimate-recovery experiments are screening samples. The results should therefore be reported by framework, model, and payload, rather than as one pooled model-independent rate.

Snapshot disclosure.. The OpenCode Go catalog records aliases, ownership, and one common `created` timestamp. Because that timestamp is identical across the catalog entries, we interpret it as catalog publication time, not an immutable checkpoint identifier. The service documentation specifies `/responses` for GPT-5.6 Luna, whereas the historical GPT artifacts were created through `/chat/completions`. A current one-request probe was blocked with HTTP 403/edge code 1010 on both routes, so it cannot establish protocol equivalence. Billing records, endpoint metadata, and local raw logs prove that calls were made, but cannot prove the exact backend checkpoint. Three public Hugging Face repositories are recorded as snapshot candidates, but no provider alias-to-revision mapping is disclosed. Exact snapshot reproducibility is consequently an explicit limitation.

11.3. Construct Validity

Trampoline construct.. The trust-escalation trampoline is operationalized as the conjunction of two source-level criteria: absence of a provenance distinction and application of a high-trust recovery frame. Partial cases are retained rather than forced into the binary label. The code excerpts, fixed commits, written rubric, and deterministic checks are preserved. This is a single-coder qualitative analysis; independent re-coding and formal disagreement adjudication remain limitations rather than completed claims.

Payload coverage.. Five payload variants are used in the canonical Reflexion matrix and native framework runs. They cover the same semantic directive with different surface forms, but cannot represent the full space of adaptive or multilingual attacks. The defense-aware screen is single-round; a multi-round attacker that learns from rejections could achieve a different rate.

Legitimate recovery.. The legitimate-recovery screen contains 100 deterministic tasks across seven failure categories, with 90 quality-passing recoveries and zero false positives. It is reported as screening evidence only. The accompanying power analysis is prospective planning information, not post-hoc confirmation.

11.4. Conclusion Validity

All reported intervals are Wilson 95% intervals, and matched binary comparisons use two-sided Fisher exact tests. The contrast and defense-aware screens are explicitly exploratory or descriptive where power is insufficient. The power-analysis artifact records that the small provenance contrast and the observed defense contrast would require substantially larger samples for confirmatory inference. We therefore avoid claims that provenance tagging or the completed defense screen has a proven population-level effect.

11.5. Reliability

The replication package retains raw per-trial logs, canonical manifests, endpoint metadata, artifact hashes, the GPT endpoint probe, the model-snapshot audit, the power analysis, the provenance manifest, the source-code audit, and the number-generation script. Credentials are not written to these artifacts. All numerical tables in this submission are generated from the JSON logs; any older draft section that cannot be traced to those logs is excluded from the quantitative claims.

Structural-defense scope.. The ReauthorizationGate section specifies a useful authorization invariant, but this paper does not evaluate a native-framework port with raw logs. Claims of zero ASR, framework-agnosticity, or negligible overhead are therefore design hypotheses and future-work requirements, not findings used to support the main RQ2/RQ3 conclusions.

12. Conclusion

This paper studies a single architectural security boundary: the recovery path of self-healing LLM agents can re-inject externally influenced failure content into a subsequent LLM turn. The resulting channel is independent of the normal forward input path. Our evidence combines source-level multiple-case analysis with native-framework execution, matched benign controls, and explicitly labeled mechanism screens.

The canonical Reflexion matrix shows substantial model heterogeneity: attack ASR ranges from 0.174 to 0.796 across five model aliases, while all matched benign trials remain at zero false positives. The native AutoGen and Aider runs demonstrate that the phenomenon is not restricted to one message-construction script: AutoGen produces a non-zero attack rate for Kimi K2.6 (0.372) with zero benign false positives, and Aider produces 0.456 attack ASR for GPT-5.6 Luna with zero benign false positives. These results support the central claim that recovery/error paths should be treated as independent trust boundaries, while the framework/model/payload strata show why a single pooled ASR would be misleading.

The native Reflexion provenance port is the primary RQ3 provenance result: raw recovery produces 0.776 attack success while TrustOrigin framing produces 0.548, with zero matched benign false positives and Fisher's exact $p = 9.78 \times 10^{-8}$. This is single-model evidence that provenance framing reduces, but does not eliminate, recovery-path obedience. The older Reflexion contrast screen is intentionally modest:

raw-versus-tagged recovery content differs by only 0.010 in pooled ASR (0.780 versus 0.770), and recovery-versus-forward differs by 0.060 (0.780 versus 0.840). We therefore report these cells as exploratory mechanism screening, not as confirmatory estimates of a large provenance effect. Similarly, the supplementary defense-aware screen changes ASR from 0.930 to 0.860, but Fisher's exact test is not significant ($p = 0.1652$). The legitimate-recovery screen obtains 90/100 successful repairs and zero false positives; it is useful as a screening result.

The design implication is consequently two-layered. Provenance marking and untrusted framing are necessary controls because they reduce the chance that an error payload is interpreted as an instruction. They should not be treated as an absolute guarantee, especially when the model is strongly instruction-following. Privileged recovery actions require an additional structural reauthorization check at the recovery sink; this is a complementary design rule rather than a replacement for content handling.

Finally, the OpenCode Go catalog, billing provenance, local raw logs, and endpoint metadata are preserved in the replication package. The catalog does not expose an immutable provider checkpoint, and the current endpoint probe was blocked with HTTP 403/1010 on both the documented GPT Responses route and the historical Chat Completions route. We therefore make no claim that provider aliases are fixed official snapshots. Exact backend-version reproducibility remains an explicit limitation and a priority for future replication.

13. Ethics Statement

13.1. Responsible Vulnerability Handling

This study evaluates a recovery-channel prompt-injection pattern using synthetic payloads and canary behavior. The load-bearing results are obtained in isolated test environments from fixed framework commits; they do not target live user data, production credentials, or third-party services beyond the model calls needed for the experiment.

Maintainer communication.. The Reflexion, AutoGen, and Aider observations are reported with different evidence labels. Reflexion has a native end-to-end recovery-path result; AutoGen and Aider have native-framework runs with matched benign controls but smaller cells; source-only observations for the remaining cases are not presented as confirmed vulnerabilities. Before public release, maintainers will receive the relevant fixed-commit excerpts, reproduction instructions, and the provenance-tagging recommendation. Communication status and links will be added to the replication package when available; no unverified acknowledgement is claimed in this manuscript.

13.2. Responsible Release

The released artifacts contain synthetic payloads that exercise the provenance property of the recovery path, the canary detector, raw trial summaries, and the corresponding remediation guidance. They do not include model-specific jailbreaks, user data, secrets, or instructions for attacking unrelated deployments. API keys are excluded from scripts, logs, manifests, and the paper source.

Model-provider use.. The experiments use provider-mediated model aliases under the provider's published service terms. The provenance manifest records endpoint, protocol, run date, model alias, billing/local-log references, and the fact that exact backend snapshots are not exposed. The endpoint verification probe was stopped after the provider edge returned HTTP 403/1010 on both the documented and historical GPT routes; no repeated probing or quota-avoidance behavior was used.

Broader impact.. Recovery loops are intended to improve reliability, but they can also move untrusted content into a context that the model treats as authoritative. We hope the evidence encourages explicit provenance tracking and authorization checks at recovery sinks. Because content-level defenses remain probabilistic and the structural gate is retained only as a design implication in this freeze, the paper avoids presenting either control as a universal solution.

14. Statements and Declarations

14.1. Competing Interests

The authors declare that they have no competing interests.

14.2. Funding

This study received no targeted external funding.

14.3. Data Availability

The accompanying replication package contains the derived trial records, canonical manifests, source-audit outputs, endpoint verification, snapshot audit, power analysis, and the scripts that generate the numerical tables. Provider response bodies and credentials are not redistributed: the former are subject to provider retention and access terms, and the latter are never included in the package. The package instead preserves local-log references, artifact hashes, protocol metadata, and the derived records needed to verify the reported counts.

14.4. Use of Artificial Intelligence Tools

An AI assistant was used during drafting, code organization, and language editing. The authors reviewed the experiments, analyses, and final manuscript, remain accountable for all claims, and approved the submitted version.

References

Aider-AI, 2025. Aider: Ai pair programming in your terminal. https://github.com/Aider-AI/aider. Production-deployed coding agent (∼3.4k stars). The `editblock_coder.apply_edits` retry path reads the target file from disk and embeds its content (which may be an adversarial file the agent was asked to edit) in a `ValueError` message that is re-injected into the next LLM iteration via the `num_reflections` loop, with no provenance tag.

All-Hands-AI, 2025. OpenHands (formerly OpenDevin): An open platform for AI software developers. https://github.com/All-Hands-AI/OpenHands. Production-deployed autonomous coding agent (∼60k stars). Tool exceptions are wrapped in `ErrorObservation` objects appended to `state.history`; the `CodeActAgent` converts the entire history (including `ErrorObservations`) into LLM messages via `conversation_memory.process_events`, with no provenance tag distinguishing error observations from successful observations.

Bird, C., Zimmermann, T., 2015. Mining software repositories in the GitHub era, in: Proceedings of the 12th Working Conference on Mining Software Repositories (MSR), pp. 354–355. doi:10.1109/MSR.2015.44.

Bovet, D.P., Cesati, M., 2005. Understanding the Linux Kernel. 3rd ed., O'Reilly Media.

Debenedetti, E., Shumailov, I., Fan, T., Hayes, J., Carlini, N., Fabian, D., Kern, C., Shi, C., Terzis, A., Tramèr, F., 2025. Defeating prompt injections by design. arXiv preprint arXiv:2503.18813 Proposes CaMeL, a capability-based defense that extracts control and data flows from a trusted query and enforces security policies when tools are called, preventing untrusted data from impacting program flow on the forward execution path.

Glaser, B.G., Strauss, A.L., 1967. The Discovery of Grounded Theory: Strategies for Qualitative Research. Aldine. Foundational text introducing grounded theory methodology, including the open coding, axial coding, and selective coding procedures used in taxonomy construction.

Gou, Z., Shao, Z., Gan, Y., Akihara, J., Qiu, X., Li, Y., Liu, P., Matsuo, Y., 2024. CRITIC: Large language models can self-correct with tool-interactive critiquing, in: International Conference on Learning Representations (ICLR).

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M., 2023. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. arXiv preprint arXiv:2302.12173 .

Hines, K., Lopez, G., Hall, M., Zarfati, F., Zunger, Y., Kiciman, E., 2024. Defending against indirect prompt injection attacks with spotlighting, in: arXiv preprint arXiv:2403.14720.

Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Zhang, C., Wang, J., Wang, Z., Yau, S.K.S., Lin, Z., et al., 2023. MetaGPT: Meta programming for a multi-agent collaborative framework. arXiv preprint arXiv:2308.00352.

Kitchenham, B.A., Pfleeger, S.L., Pickard, L.M., Jones, P.W., Hoaglin, D.C., El-Emam, K., Rosenberg, J., 2002. Preliminary guidelines for empirical research in software engineering. IEEE Transactions on Software Engineering 28, 721–734. doi:10.1109/TSE.2002.1027796.

LangChain, 2024. LangGraph: Building stateful, multi-actor applications with LLMs. https://github.com/langchain-ai/langgraph.

Liu, Y., Jia, Y., Geng, R., Jia, J., Gong, N.Z., 2024. Formalizing and benchmarking prompt injection attacks and defenses, in: USENIX Security Symposium. ArXiv:2310.12815.

Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., Gupta, S., Majumder, B.P., Hermann, K., Welleck, S., Yazdanbakhsh, A., Clark, P., 2023. Self-refine: Iterative refinement with self-feedback, in: Advances in Neural Information Processing Systems (NeurIPS).

OWASP, 2025. OWASP top 10 for LLM applications (2025 edition). https://genai.owasp.org/llmrisk/. Industry-standard taxonomy of LLM application risks. *LLM01: Prompt Injection* covers forward-path injection; the recovery-channel surface studied in this paper is not enumerated.

Runeson, P., Höst, M., 2009. Guidelines for conducting and reporting case study research in software engineering. Empirical Software Engineering 14, 131–164. doi:10.1007/s10664-008-9102-8. foundational guidelines for case study research in software engineering, including research design, data collection, and validity threats.

Saltzer, J.H., Schroeder, M.D., 1975. The protection of information in computer systems. Proceedings of the IEEE 63, 1278–1308.

Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., Yao, S., 2023. Reflexion: Language agents with verbal reinforcement learning, in: Advances in Neural Information Processing Systems (NeurIPS).

Silberschatz, A., Galvin, P.B., Gagne, G., 2003. Operating System Concepts. 7th ed., Wiley.

Storey, M.A., 2008. Examining software documentation research. Empirical Software Engineering 13, 319–331.

Strauss, A.L., Corbin, J.M., 1998. Basics of Qualitative Research: Techniques and Procedures for Developing Grounded Theory. 2nd ed., Sage Publications. Refined grounded theory coding procedures (open, axial, selective) applied in this paper's taxonomy construction.

Wallace, E., Stern, M., Song, D., 2024. The instruction hierarchy: Training LLMs to prioritize privileged instructions, in: Proceedings of the 41st International Conference on Machine Learning (ICML).

Willison, S., 2023. Prompt injection: What's the worst that can happen? https://simonwillison.net/2023/Oct/14/prompt-injection/.

Wohlin, C., Runeson, P., Höst, M., Ohlsson, M.C., Regnell, B., Wesslén, A., 2012. Experimentation in Software Engineering. Springer. Standard reference on empirical software engineering methodology, including validity threat classification.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., et al., 2023. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. arXiv preprint arXiv:2308.08155.

Yang, J., Jimenez, C.E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., Press, O., 2024. SWE-agent: Agent-computer interfaces enable automated software engineering. arXiv preprint arXiv:2405.15793.

Zhan, Q., Fang, R., Panchal, H.S., Kang, D., 2025. Adaptive attacks break defenses against indirect prompt injection attacks on LLM agents, in: Findings of the Association for Computational Linguistics: NAACL 2025. Evaluates 8 defenses against indirect prompt injection and bypasses all of them with adaptive attacks, achieving >50% ASR. The attack targets the forward execution path (tool outputs), not the recovery channel.